

Task 3: AIEA Nautilus Setup

Rami Al Habash 1/21/2026

CMPM 118 Dr. Leilani Glippin

Objective

- Setting up Nautilus.
- Testing out Nautilus Jupyterhub and Kubernetes guides.
- Creating a PVC, deployment, and job.

Setup

I had to ask on Discord to be added to the auditors namespace on Nautilus. I took my time figuring things out, and I looked at some of the guides.

Jupyter Lab/Kubernetes

I tried running the file on the task straight on Jupyter Lab, and it worked. I had no issues. It was nice to get an understanding of the type of files I'll be running in this lab. I then pivoted to some starter guides using Kubernetes. I learned how to set my namespace as the primary namespace. I also learned how to use important commands like "kubectl get nodes/pvc/job/deployment/etc". The guides on the site were extremely valuable for me to get comfortable with Nautilus and navigate it using Kubernetes.

Creating PVC, Deployment, and Job

I used the templates in the GitHub repo to help me set up the YAML files for a PVC, deployment, and Job. I understand their functionalities well. The PVC is more of a storage that stores your data when you run jobs and deployments, so it's helpful to always use it. A deployment allows you to create your own pod that you can use to run your files on. I like the deployment pod because it doesn't terminate after running and allows you to modify your file in the pod. You can also leave pods running for a while if you have a long job. Jobs are for quicker one-and-done tasks. The pod terminates once the job is done. They log your file results, so you have to use a kubectl command to access your results. Jobs aren't meant to last for long. Jobs are meant for quicker run times. I tested out training the model in the file [cifar10train.py](#) using a deployment and a job. I logged my results below on page 2.

Conclusion

I have learned a lot about training models using shared resources on Nautilus. This will help me when I have larger models I want to train. Using the resources on Nautilus should be a lot quicker than training models natively on my laptop. I'm glad I have everything set up, and I'm ready to use Nautilus whenever. I am now ready to move on to the next task.

Results

Deployment

```
root@ralhabas-task3-dep-6c65f5dfd6-wf8cd:/pvcvolume# python3 cifar10train.py
Files already downloaded and verified
Files already downloaded and verified
Device: cuda:0
[1, 2000] loss: 2.177
[1, 4000] loss: 1.850
[1, 6000] loss: 1.660
[1, 8000] loss: 1.550
[1, 10000] loss: 1.513
[1, 12000] loss: 1.479
[2, 2000] loss: 1.381
[2, 4000] loss: 1.367
[2, 6000] loss: 1.343
[2, 8000] loss: 1.302
[2, 10000] loss: 1.291
[2, 12000] loss: 1.268
Finished Training
Accuracy of the network on the 10000 test images: 55 %
Accuracy for class: plane is 54.1 %
Accuracy for class: car is 66.1 %
Accuracy for class: bird is 45.4 %
Accuracy for class: cat is 53.2 %
Accuracy for class: deer is 44.4 %
Accuracy for class: dog is 41.2 %
Accuracy for class: frog is 63.1 %
Accuracy for class: horse is 59.1 %
Accuracy for class: ship is 77.0 %
Accuracy for class: truck is 51.6 %
Total time: 94.87 seconds
root@ralhabas-task3-dep-6c65f5dfd6-wf8cd:/pvcvolume#
```

Job

```
● ramialhabash@eduroam-169-233-210-162 Nautilus % kubectl logs ralhabas-task3-job-9vhrt

Files already downloaded and verified
Files already downloaded and verified
Device: cuda:0
[1, 2000] loss: 2.200
[1, 4000] loss: 1.841
[1, 6000] loss: 1.643
[1, 8000] loss: 1.559
[1, 10000] loss: 1.488
[1, 12000] loss: 1.451
[2, 2000] loss: 1.392
[2, 4000] loss: 1.363
[2, 6000] loss: 1.334
[2, 8000] loss: 1.298
[2, 10000] loss: 1.279
[2, 12000] loss: 1.263
Finished Training
Accuracy of the network on the 10000 test images: 54 %
Accuracy for class: plane is 41.4 %
Accuracy for class: car is 63.2 %
Accuracy for class: bird is 46.4 %
Accuracy for class: cat is 31.9 %
Accuracy for class: deer is 49.1 %
Accuracy for class: dog is 63.5 %
Accuracy for class: frog is 51.8 %
Accuracy for class: horse is 66.9 %
Accuracy for class: ship is 59.5 %
Accuracy for class: truck is 70.5 %
Total time: 108.97 seconds
```